

Module: AI-Driven Product Backlog & Artifact Creation



Lecture 1: Generating a Product Backlog from a Vision

This lecture focuses on translating a high-level idea into a tangible starting point for an Agile project.

Step 1: Deep Research (using Perplexity)

- **Goal:** To gather market, competitor, and user data to inform the product backlog.
- **Input Document Required:** Project Vision Document (A 1-2 page document detailing the product idea, target audience, problem to be solved, and unique value proposition).

Prompt in COSTAR Format:

- **C (Context):** You are a senior market research analyst and product strategist. I am the Product Owner for a new software product. I am providing you with our initial project vision document.
[Paste the content of your Project Vision Document here]
- **O (Objective):** Your objective is to perform a deep-dive analysis of the market landscape for this product. Specifically, identify:
 1. Key market trends and user expectations in this domain.
 2. The top 3 direct and indirect competitors and list their core features.
 3. Potential challenges, risks, or technological hurdles we might face.
 4. Key features and functionalities that are considered "table stakes" (must-haves) for a product like this.
- **S (Style):** Present your findings in a structured report format. Use clear headings for each section (e.g., "Market Trends," "Competitor Analysis," "Potential Risks," "Table Stakes Features"). Use bullet points for lists.
- **T (Tone):** Analytical, data-driven, and professional.
- **A (Audience):** The output is for the internal product development team and key stakeholders to help us make informed decisions.
- **R (Response):** A well-organized report in markdown format.

Step 2: Generating the Initial Product Backlog (using ChatGPT/Gemini)

- **Goal:** To convert the vision and research into a structured, prioritized product backlog.
- **Input Documents Required:**
 1. Project Vision Document
 2. AI-Generated Research Report (the output from Step 1)
 3. User Personas Document (A document describing 2-3 fictional target users, their goals, and pain points).

Prompt in COSTAR Format:

- **C (Context):** You are an expert Agile Product Owner with 15 years of experience building software. You are tasked with creating the initial product backlog for a new project. I will provide you with the project vision, a market research report, and our user personas.
 - 1) Refer to the Project Vision and Goals Document

- 2) Refer to the Research Report
 - 3) Refer to the User Personas Document
 - **O (Objective):** Your goal is to generate a comprehensive product backlog. Break down the product idea into a list of Epics. Under each Epic, create several high-level user stories. For each user story, identify the primary user persona it benefits and suggest an initial priority using the MoSCoW method (Must-have, Should-have, Could-have, Won't-have).
 - **S (Style):** Structure the output as a markdown table with the following columns: "Epic," "User Story (As a..., I want..., so that...)," "Associated Persona," and "MoSCoW Priority."
 - **T (Tone):** Practical, concise, and user-centric.
 - **A (Audience):** The Scrum team (Developers, QA, Scrum Master) who will begin refining this backlog.
 - **R (Response):** A markdown table containing the initial product backlog.
-



Lecture 2: Crafting Epics and User Stories with AI

This lecture demonstrates how to refine high-level backlog items into actionable user stories with clear requirements.

Step 1: Decomposing an Epic into User Stories with Acceptance Criteria

- **Goal:** To break down a large feature (Epic) into smaller, well-defined user stories complete with acceptance criteria.
- **Input Documents Required:**
 1. Initial Product Backlog (the output from Lecture 1)
 2. User Personas Document

Prompt in COSTAR Format:

- **C (Context):** You are an experienced Agile coach specializing in story writing. We are in a backlog refinement session. I will provide you with one Epic from our product backlog and our relevant user persona(s).
Epic: Refer to the attached sheet and focus on the "Cart and Checkout" Epic.
Relevant Persona: Refer to the attached User Persona document.
 - **O (Objective):** Your task is to decompose this Epic into an exhaustive set of smaller, well-formed user stories. Each user story must follow the standard **As a [persona], I want [action], so that [benefit]** format. For EACH user story, you must also generate a list of 3-5 detailed acceptance criteria using the **Gherkin Given-When-Then** format.
 - **S (Style):** Use a hierarchical list. The Epic should be the top-level item. Each user story should be a sub-item. The acceptance criteria for each story should be nested bullet points under it.
 - **T (Tone):** Detailed, precise, and unambiguous.
 - **A (Audience):** The development and QA team who will use this to build and test the feature.
 - **R (Response):** A structured markdown list detailing the epic, its user stories, and their corresponding acceptance criteria.
-

✓ Lecture 3: Defining "Done" and "Ready" with AI Assistance

This lecture shows how AI can help create foundational team agreements that improve workflow and quality.

Step 1: Generating a Definition of Done (DoD)

- **Goal:** To create a shared understanding of what it means for work to be complete.
- **Input Document Required:** [Team Skill Matrix](#) (A brief document outlining the team's technology stack (e.g., React, Node.js, AWS), project type (e.g., Mobile App, B2B SaaS), and any known standards (e.g., WCAG 2.1 for accessibility)).

Prompt in COSTAR Format:

- **C (Context):** You are a seasoned Scrum Master setting up a new Agile team. I will provide you with our team's context.
Refer to the attached Skill Matrix document.
 - **O (Objective):** Generate a comprehensive and robust **Definition of Done (DoD)** for the user story - "US1 — Add to Cart from PDP/PLP". The DoD should be a checklist of all the activities that must be completed before a story can be considered "done." Include items related to code quality, testing, documentation, and release readiness.
 - **S (Style):** A markdown checklist, categorized by Agile activity (e.g., "Development," "Testing & QA," "Documentation," "Review & Merge").
 - **T (Tone):** Authoritative, clear, and prescriptive.
 - **A (Audience):** The entire Agile development team.
 - **R (Response):** A markdown checklist representing the team's Definition of Done.
-

Step 2: Generating a Definition of Ready (DoR)

- **Goal:** To create a checklist ensuring backlog items are well-prepared for a sprint.
- **Input Document Required:** [Skill Matrix Document](#).

Prompt in COSTAR Format:

- **C (Context):** You are a Scrum Master facilitating a discussion between the Product Owner and the development team. I am providing our team's context.
Refer to the attached Skill Matrix document.
 - **O (Objective):** Generate a clear and concise **Definition of Ready (DoR)**. This should be a checklist that the Product Owner uses to ensure a user story is ready to be pulled into a sprint. It should focus on clarity, testability, and shared understanding.
 - **S (Style):** A simple, easy-to-follow markdown checklist.
 - **T (Tone):** Collaborative and clear, acting as a team agreement.
 - **A (Audience):** The Product Owner and the development team.
 - **R (Response):** A markdown checklist representing the team's Definition of Ready.
-



Lecture 4: Generating Acceptance Test Cases

This lecture demonstrates how to accelerate the testing process by generating test cases directly from a user story.

Step 1: Generating Test Cases and Reasoning

- **Goal:** To create specific test cases from a user story and its acceptance criteria, including the rationale behind each test.
- **Input Document Required:** A Single User Story with Acceptance Criteria (Select one complete user story generated in Lecture 2).

Prompt in COSTAR Format:

- **C (Context):** You are a highly skilled Senior QA Automation Engineer. Your task is to write test cases for a user story that has just been refined and marked as "Ready." I will provide the user story and its acceptance criteria.
[Paste the user story and its Gherkin-formatted acceptance criteria here]
- **O (Objective):** Based on the provided story and acceptance criteria, generate a comprehensive set of acceptance test cases. For each test case, include a "Reasoning" column that explicitly explains which part of the acceptance criteria or user story the test is validating and why it's important (e.g., "This tests the primary success path," "This is a negative test for invalid input," "This validates the edge case where...").
- **S (Style):** A detailed markdown table with the columns: "Test Case ID," "Test Scenario Description," "Test Steps," "Expected Result," and "Reasoning."
- **T (Tone):** Meticulous, logical, and analytical.
- **A (Audience):** Manual QA testers, automation engineers, and developers who need to understand how the feature will be validated.
- **R (Response):** A markdown table containing the detailed test cases.

Step 2: Formatting the Output into a Document

- **Goal:** To convert the generated markdown into a shareable document format.
- **Input Document Required:** Generated Test Cases Table (the markdown output from the previous step).

Simple Follow-up Prompt:

"Now, take the markdown table of test cases you just created. Format this information into a professional-looking document. Start with a title: 'Acceptance Test Cases'. Then, restate the user story and its acceptance criteria. Finally, present the test cases in a clean, well-formatted table."